DEEP LEARNING

Complete Reference Notes

From a Single Neuron to Neural Networks that Learn

A beginner-friendly, story-style guide covering fundamentals, the perceptron, forward & backward propagation, activation functions, loss functions, and optimizers.

Prepared for future reference · Somnath Singh

How to Read These Notes

These notes are written as one continuous story. Each idea is a stepping stone to the next, so it is best to read them in order the first time. Later, you can jump to any part using the contents below. Every technical word is explained in plain language before it is used, and the diagrams and formulas are drawn exactly the way they appear in the original handwritten notes.

> **The big story in one line**
>
> We want a computer to **learn from examples**. To do that we build a network of tiny decision units (**neurons**), let it **guess** an answer (forward propagation), **measure how wrong** the guess is (loss function), and then **nudge the network** to be a little less wrong next time (backward propagation + optimizers). Activation functions are what let the network learn *complex*, non-straight-line patterns.

Table of Contents

PART 1

Foundations — What Deep Learning Really Is

Before touching any maths, let us understand where deep learning sits and why the whole world suddenly started talking about it.

1.1 Three circles: AI, Machine Learning, Deep Learning

Think of three circles sitting inside one another. The largest is **Artificial Intelligence (AI)** — the broad dream of making machines behave intelligently. Inside it is **Machine Learning (ML)** — machines that learn patterns from data instead of being programmed with fixed rules. Inside ML sits **Deep Learning (DL)**, our topic. Deep learning tries to **mimic the human brain** using **multi-layered neural networks** — many small units connected in layers, just like neurons in our head.

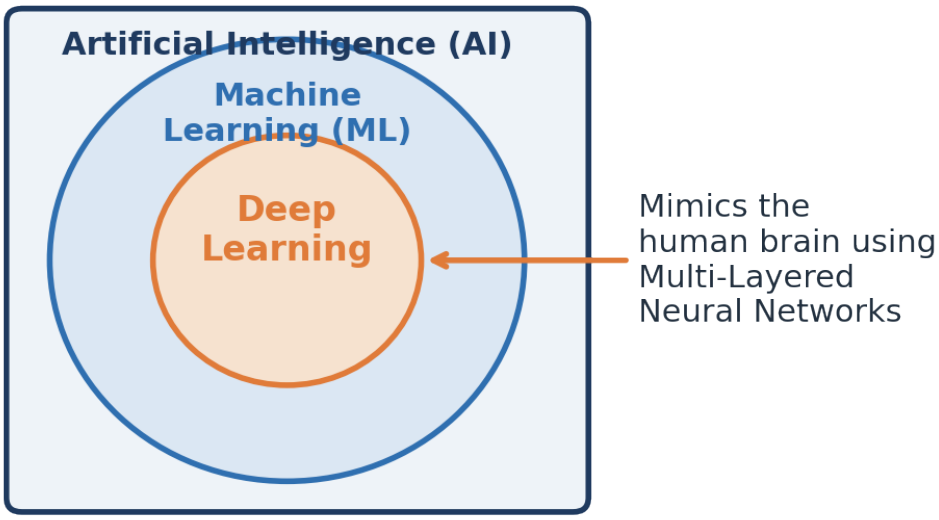


Figure 1.1 — Deep Learning is a specialised part of Machine Learning, which itself is a part of AI.

> **Key idea**
> The word **"deep"** simply means the network has **many layers** stacked one after another. More layers let the model understand more complex patterns.

1.2 The three main families of neural networks

Deep learning is not a single tool — it is a family. Depending on the kind of data you have, you pick a different type of network. The notes highlight three:

Network	Full form	Best used for	Popular examples
ANN	Artificial Neural Network	Classification & Regression on normal (tabular) data	Pass/Fail prediction
CNN	Convolutional Neural Network	Images and video frames (Computer Vision)	R-CNN, Mask R-CNN, YOLO v5/v6/v7
RNN	Recurrent Neural Network	Text and time-series (sequence data)	LSTM, GRU, Transformers, BERT

In simple words: use an **ANN** when your data is a plain table of numbers, a **CNN** when your input is an image, and an **RNN** (or its modern cousins like Transformers and BERT) when your input is a

sequence such as a sentence or a series of readings over time. In this document we focus on the **ANN**, because understanding it teaches the ideas that power all the others.

1.3 Why did deep learning suddenly become popular?

Neural networks are an old idea, so why the recent excitement? Around 2005–2012 something changed. Social media platforms (Facebook, YouTube, WhatsApp, LinkedIn, Twitter) started generating **data exponentially** — images, text, documents and videos — the era of **Big Data**. Two things then came together:

- **Cheaper, faster hardware.** GPUs (mainly from NVIDIA) became affordable, and GPUs are brilliant at training neural networks quickly.
- **Huge amounts of data.** Deep learning models are hungry — the more data they get, the better they perform, unlike traditional ML which flattens out early.

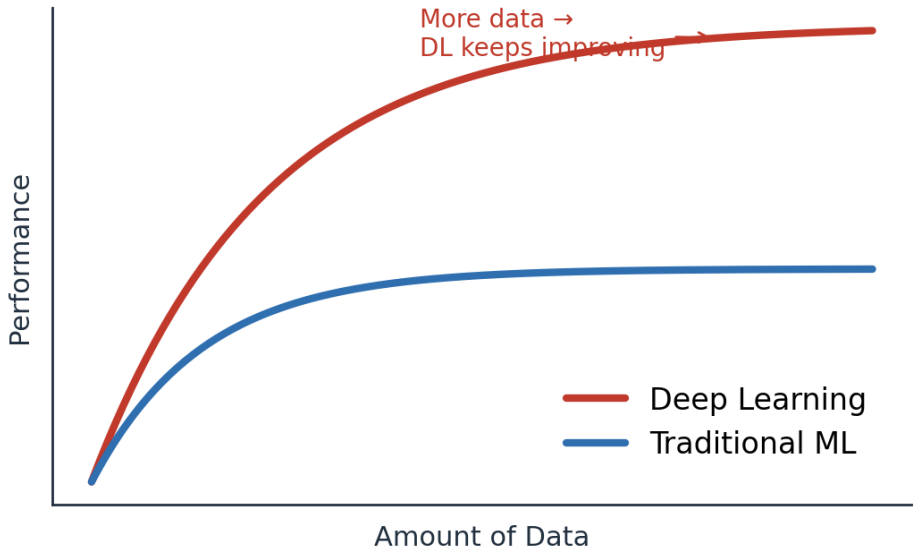


Figure 1.2 — With more data, deep learning keeps improving while traditional ML plateaus.

Because of this, deep learning is now used across **medical, e-commerce, retail, marketing** and many more domains. Open-source **frameworks** also lowered the barrier: **TensorFlow** (from Google) and **PyTorch** (from Facebook/Meta, popular in research) let anyone build end-to-end projects, and their large communities keep them growing.

> **Bridge to Part 2**
> We now know *what* deep learning is and *why* it matters. Next comes the natural question: what is the smallest piece a neural network is built from? Meet the **perceptron** — a single artificial neuron.

PART 2

The Perceptron — The Building Block

A neural network is just many tiny units working together. If you understand one unit, you understand the whole network. That unit is the perceptron.

2.1 What is a perceptron?

A **perceptron** is a single **artificial neuron** — the smallest neural network unit. It is also called a **single-layer neural network** and it works as a **binary classifier** (it answers questions with two possible outcomes, like Pass/Fail or Yes/No). It has four parts: an **input layer**, **weights**, a **bias**, and an **activation function**.

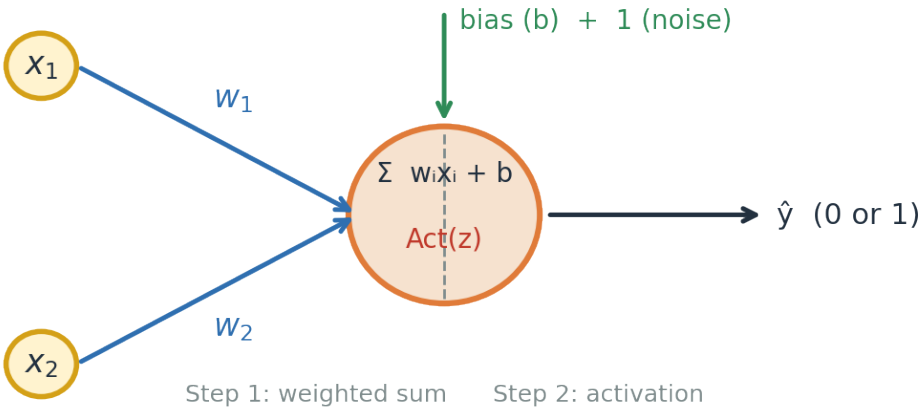


Figure 2.1 — A single neuron: inputs are multiplied by weights, a bias is added, then an activation decides the output.

2.2 What happens inside the neuron (two steps)

Step 1 — Weighted sum. Each input x is multiplied by its own **weight** w (a weight says how important that input is). We add all of these up and add a **bias** b. The result is called **z**:

$$z = \sum_{i=1}^{n} w_i x_i + b$$

Step 2 — Activation. The number **z** can be anything. We pass it through an **activation function** that squashes it into a clean, useful output. The activation is what turns a plain sum into a decision.

2.3 Two early activation choices: Step vs Sigmoid

The very first idea was the **step function**: if z is above a threshold the neuron fires (outputs 1), otherwise 0. It is decisive but harsh — a tiny change in z can flip the answer completely. The **sigmoid** function is smoother: it turns z into a probability-like value between **0 and 1**, so we get a sense of *how* confident the neuron is, not just yes/no.

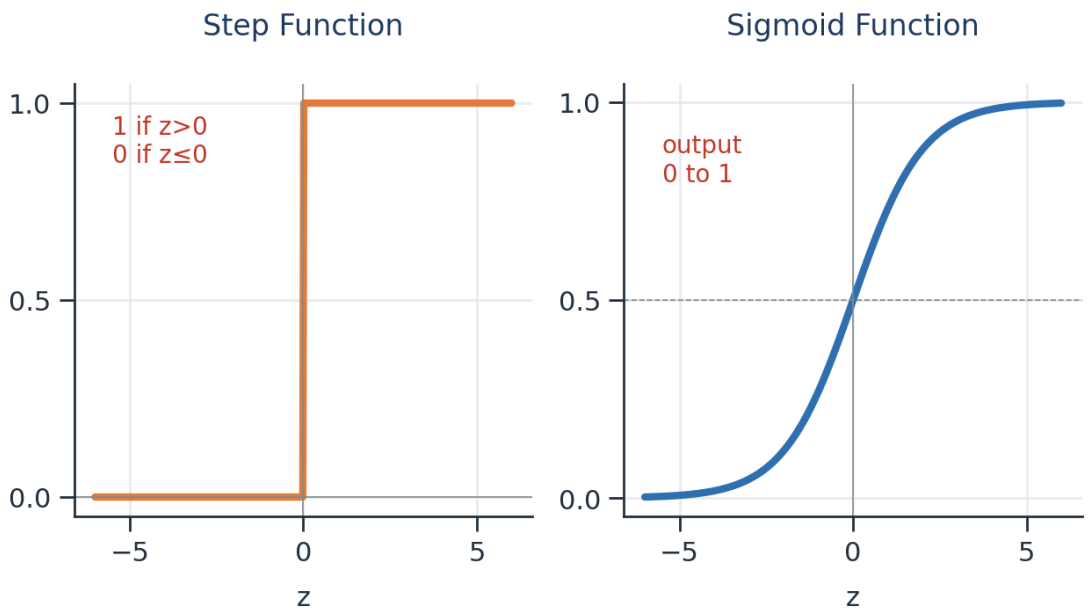


Figure 2.2 — The step function gives a hard 0/1; the sigmoid gives a smooth value between 0 and 1.

2.4 A perceptron is a straight-line (linear) classifier

Here is the important limitation. A single perceptron can only separate data with a **straight line**. If two groups of points can be split by a straight line, we call them **linearly separable**, and one perceptron is enough. But many real problems are **non-linearly separable** — no single straight line can divide them. For those, we must stack many neurons into layers, giving a **Multi-Layer Neural Network**.

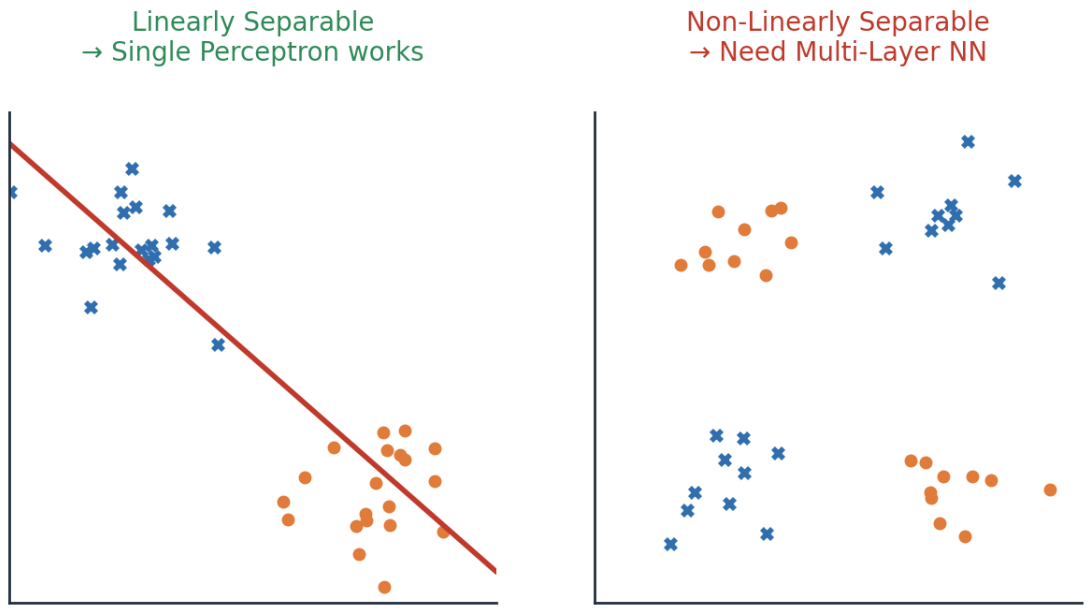


Figure 2.3 — Left: one straight line works. Right: no straight line works — we need multiple layers.

Bridge to Part 3

This single limitation is exactly why deep learning exists. By connecting many perceptrons across several layers, the network can bend and combine many straight lines into complex boundaries. That richer model is the **Multi-Layered Perceptron**, which we explore next.

PART 3

Multi-Layer Neural Networks (ANN)

Connect many neurons in layers and you get an Artificial Neural Network — the model that can learn almost any pattern. This is the heart of deep learning.

3.1 The five pillars of an ANN

A **Multi-Layered Perceptron** (the modern ANN, whose ideas were championed by **Geoffrey Hinton**) is built from five key concepts. Every remaining part of these notes is really just a deep dive into one of them:

- **Forward Propagation** — the network makes a guess.
- **Loss Function** — we measure how wrong the guess is.
- **Backward Propagation** — the error is sent back through the network.
- **Optimizers** — the weights are updated to reduce the error.
- **Activation Functions** — added at each neuron so the network can learn non-linear patterns.

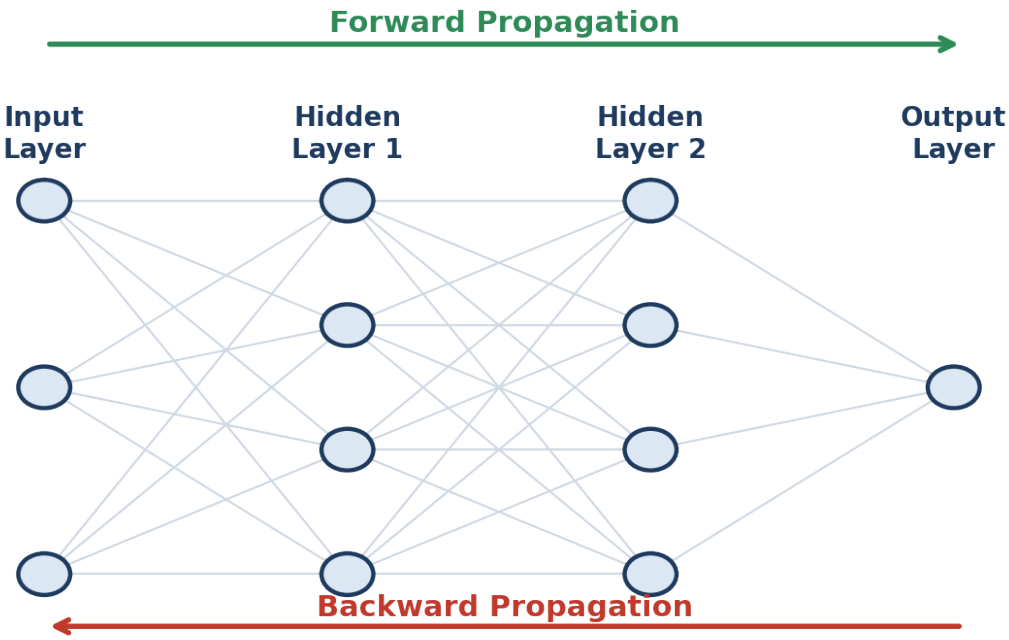


Figure 3.1 — Data flows left→right to make a prediction (forward), then the error flows right→left to learn (backward).

3.2 Forward propagation — making a guess

Forward propagation is simply Step 1 and Step 2 (from the perceptron) repeated layer by layer. Let us walk through the exact example from the notes: predicting whether a student will **Pass or Fail** using three inputs — **IQ**, **Study hours** and **Play hours**.

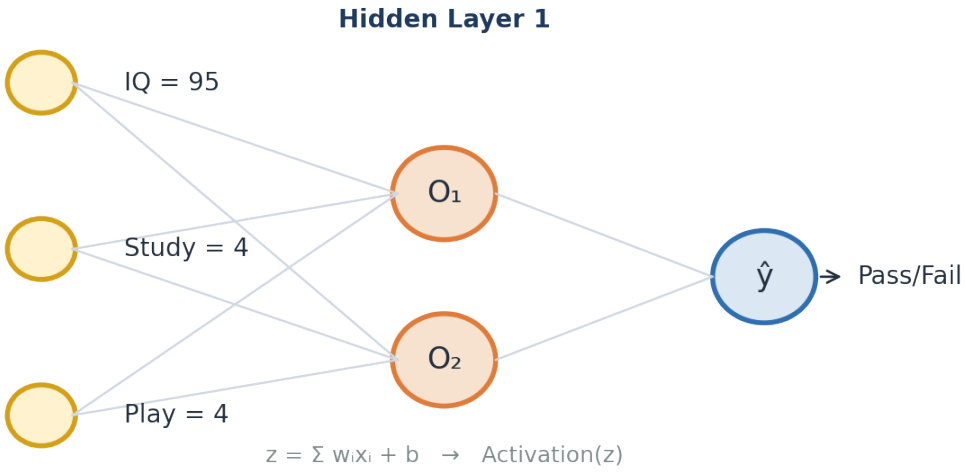


Figure 3.2 — Worked example: three inputs feed a hidden layer, which feeds the output neuron.

Take one student with IQ = 95, Study = 4, Play = 4, and weights **[0.01, 0.02, 0.03]** with bias **0.01**. In the first hidden neuron we compute z:

$$z = 95(0.01) + 4(0.02) + 4(0.03) + 1(0.01) = 1.151$$

Then we apply the sigmoid activation to squash z into a value between 0 and 1:

$$\sigma(1.151) = \frac{1}{1 + e^{-1.151}} = 0.759$$

So this neuron outputs **O■ = 0.759**. That output then becomes the input to the next layer, and we repeat the same two steps until we reach the output neuron, which produces the final prediction ■. That is the entire forward pass — multiply, add bias, activate, and pass forward.

3.3 Measuring the mistake, then learning from it

Once the network produces a prediction ■, we compare it with the true answer **y**. The gap between them is the **error**, and the **loss function** turns that error into a single number (we study loss functions in detail in Part 5). The whole goal of learning is to make this number as small as possible.

To shrink the error, the network adjusts its **weights**. But in which direction, and by how much? This is answered by **backward propagation** together with an **optimizer**, using one central formula — the **weight-update rule**:

$$W_{new} = W_{old} - \eta \frac{\partial L}{\partial W_{old}}$$

In words: the **new weight** equals the **old weight** minus the **learning rate** (η, a small number that controls step size) multiplied by the **gradient** (∂L/∂W, the slope that tells us how the loss changes when this weight changes). The bias is updated with the very same idea:

$$b_{new} = b_{old} - \eta \frac{\partial L}{\partial b_{old}}$$

3.4 Gradient descent — walking downhill to the lowest error

Picture the loss as a valley. High up the slopes the error is large; at the very bottom — the **global minima** — the error is smallest. **Gradient descent** is the strategy of taking small steps downhill, again and again, until we reach the bottom. The **slope (gradient)** tells us which way is downhill, and the **learning rate** decides how big each step is.

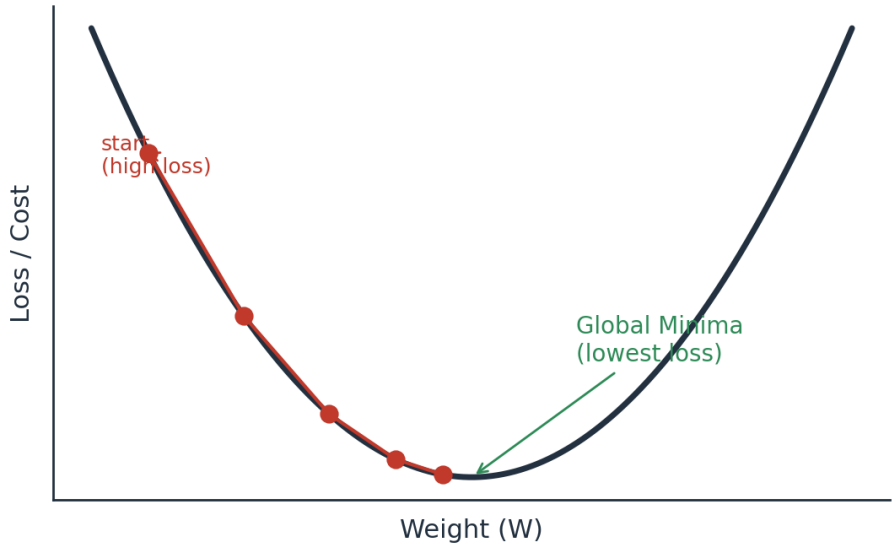
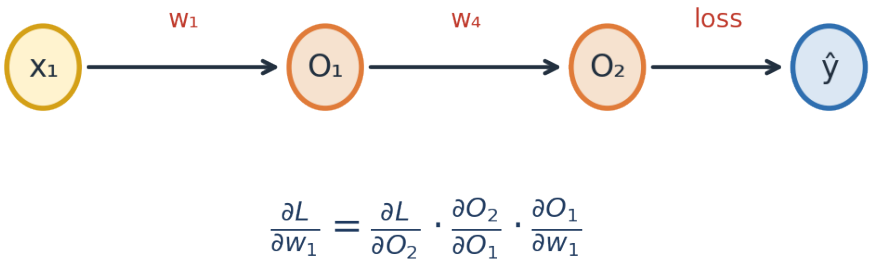


Figure 3.3 — Gradient descent takes repeated downhill steps toward the point of lowest loss (global minima).

> **Why the learning rate matters**
>
> If the learning rate is **too large**, we may leap over the bottom and never settle (it may never converge). If it is **too small**, learning is painfully slow. A common safe starting value is a small number like **0.001**. When the weight reaches the global minima, the update becomes almost zero, so W_new ≈ W_old and learning naturally stops.

3.5 The chain rule — how deep layers get their share of the blame

There is one puzzle left. The final error appears at the output, but a weight sitting deep in the network also helped cause it. How do we measure a hidden weight's effect on the final loss? The answer is the **chain rule of derivatives**. It links the loss to a deep weight by multiplying the small effects along the path from that weight all the way to the output:



$$\frac{\partial L}{\partial w_1} = \frac{\partial L}{\partial O_2} \cdot \frac{\partial O_2}{\partial O_1} \cdot \frac{\partial O_1}{\partial w_1}$$

Figure 3.4 — The chain rule multiplies the step-by-step effects to find how a deep weight influences the loss.

This is precisely what "backpropagation" means: the error is carried **backward**, and at each layer the chain rule hands every weight its fair share of responsibility, so the optimizer knows exactly how to adjust it. Keep this multiplication of small numbers in mind — it returns in Part 4 as the famous **vanishing gradient problem**.

PART 4

Activation Functions

Activation functions are the ingredient that lets a network learn curved, complex patterns instead of only straight lines. Here we meet each one, with its strengths and weaknesses.

4.1 Why do we even need them?

Without an activation function, every neuron would just do a weighted sum — and stacking many sums still gives only a straight-line relationship. The **activation function adds the "bend"**, letting the network model the non-linear patterns we saw in Part 2. It also controls the range of a neuron's output. Let us look at the whole family, in the order they were developed.

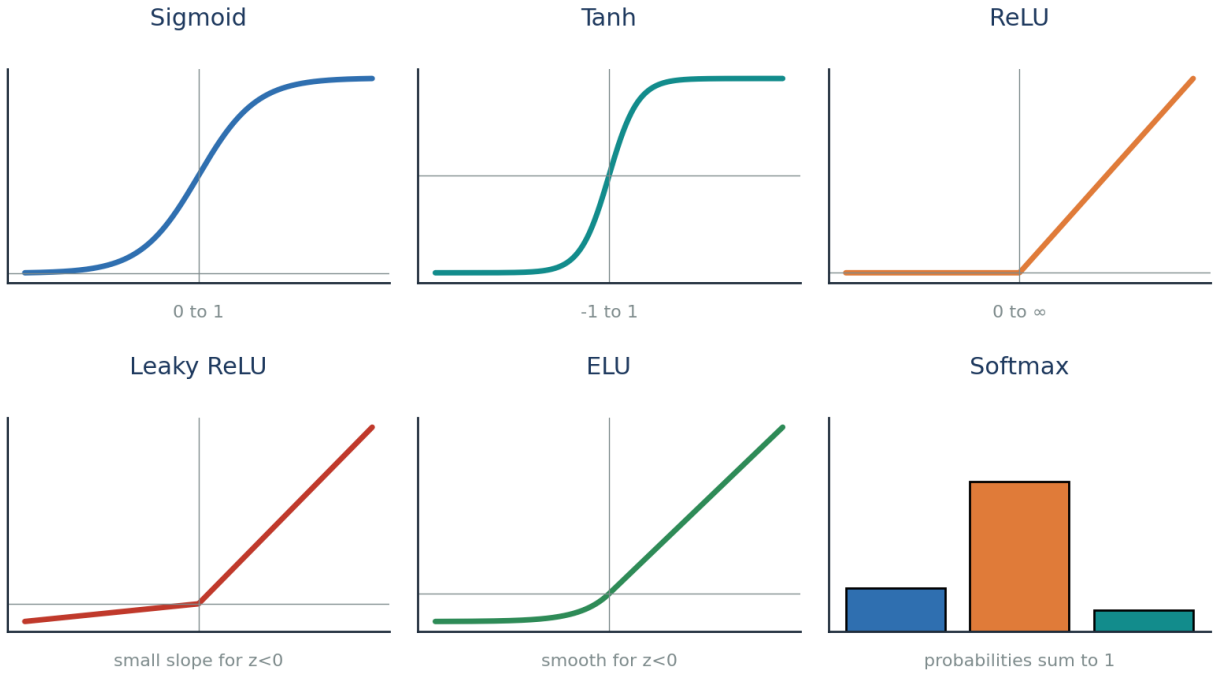


Figure 4.1 — The activation-function family at a glance.

4.2 Sigmoid (output range 0 to 1)

$$\sigma(z) = \frac{1}{1 + e^{-z}}$$

The sigmoid squashes any number into the range **0 to 1**, which makes it perfect for **binary classification** output, where we want a probability. Its derivative, however, is small (only 0 to 0.25), and that becomes a problem in deep networks.

✓ Advantages

- Great for binary classification
- Gives a clear, probability-like output near 0 or 1

✗ Disadvantages

- Prone to the vanishing gradient problem
- Output is not zero-centred → weight updates are less efficient
- Maths (exponentials) is relatively slow

4.3 Tanh (output range –1 to 1)

$$\tanh(z) = \frac{e^z - e^{-z}}{e^z + e^{-z}}$$

Tanh looks like the sigmoid but is **zero-centred** (its output spans –1 to 1). Being zero-centred makes weight updates more efficient, which is why tanh is usually preferred over sigmoid in hidden layers.

✓ Advantages	✗ Disadvantages
• Zero-centred → more efficient weight updates • Stronger gradients than sigmoid	• Still prone to the vanishing gradient problem • Still computationally heavier (uses exponentials)

4.4 ReLU — Rectified Linear Unit

$$f(z) = \max(0,\ z)$$

ReLU is beautifully simple: if the input is positive, keep it; if negative, output 0. Because it is just a comparison, it is **extremely fast** and largely **solves the vanishing gradient problem** for positive values (its derivative is a clean 1). This made ReLU the default choice for hidden layers.

But it has a catch. If a neuron's input is always negative, ReLU always outputs 0, its derivative is 0, and the weight-update formula leaves the weight unchanged (W_new ≈ W_old). That neuron effectively **dies** and stops learning — the **Dead Neuron / Dying ReLU problem**.

✓ Advantages	✗ Disadvantages
• Solves vanishing gradient (positive side) • Very fast — just max(0, z) • Faster than sigmoid or tanh	• Dead Neuron problem (dies for negative inputs) • Output is not zero-centred

4.5 Leaky ReLU and Parametric ReLU

$$f(z) = \max(0.01z,\ z)$$

To cure the dead neuron, **Leaky ReLU** lets a tiny slope through for negative inputs (e.g. 0.01·z) instead of flat zero — so the neuron always has a small gradient and never fully dies. **Parametric ReLU** is the same idea, but the small slope (α, a hyperparameter like 0.01, 0.02, 0.03) is **learned** rather than fixed.

✓ Advantages	✗ Disadvantages
• Keeps all the benefits of ReLU • Removes the Dead ReLU problem (neurons stay alive)	• Still not perfectly zero-centred

4.6 ELU — Exponential Linear Unit

$$f(z) = z \ \ (z > 0)\ ; \quad \alpha(e^z - 1) \ \ (z \le 0)$$

ELU keeps positive values as-is but uses a smooth exponential curve for negative values. This makes it **zero-centred** and completely avoids dead neurons — it was designed specifically to fix ReLU's problems.

✓ Advantages	✗ Disadvantages
• No Dead ReLU issues • Zero-centred output	• Slightly more computationally intensive (uses exponentials)

4.7 Softmax — for multi-class classification

$$\text{softmax}(z_i) = \frac{e^{z_i}}{\sum_{k=1}^{K} e^{z_k}}$$

Everything so far handled a single output. But what if we must choose among **many** classes — say Cat, Dog, Monkey, Horse? **Softmax** sits at the output layer and converts the scores into **probabilities that add up to 1**. The class with the highest probability is the network's answer. In short: use **sigmoid** for binary output, and **softmax** for multi-class output.

4.8 The vanishing gradient problem — why activation choice matters

Remember the chain rule from Part 3: to update a deep weight we **multiply** many derivatives together. With sigmoid, each derivative is at most 0.25. Multiply several small numbers (0.25 × 0.25 × 0.25 …) and the result becomes almost **zero**. The gradient has **vanished**.

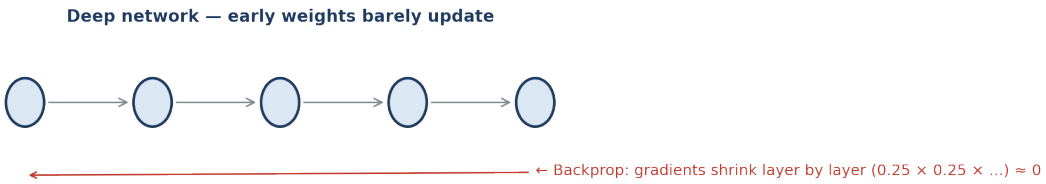


Figure 4.2 — As the error travels back through many layers, tiny derivatives multiply toward zero.

When the gradient is nearly zero, the weight-update formula gives W_new ≈ W_old — the early layers barely learn. This is the **vanishing gradient problem**, and it is the single biggest reason we moved from sigmoid/tanh to **ReLU and its variants** in hidden layers. Understanding this connects Parts 3 and 4 into one story: the maths of learning directly decides which activation function we should use.

> **Quick decision guide**
>
> **Hidden layers:** start with **ReLU**; switch to **Leaky ReLU / ELU** if neurons are dying.
> **Output — binary:** use **Sigmoid**. **Output — multi-class:** use **Softmax**.
> **Avoid** sigmoid/tanh deep inside big networks because of vanishing gradients.

4.9 Activation functions — side-by-side summary

Function	Output range	Main strength	Main weakness
Sigmoid	0 to 1	Binary output / probability	Vanishing gradient, not zero-centred
Tanh	-1 to 1	Zero-centred	Still vanishing gradient
ReLU	0 to ∞	Fast, fixes vanishing (positive)	Dead neuron problem
Leaky ReLU	small neg to ∞	No dead neurons	Not fully zero-centred
ELU	smooth neg to ∞	Zero-centred, no dead neurons	Slightly slower
Softmax	0 to 1 (sums to 1)	Multi-class output	Output layer only

PART 5

Loss & Cost Functions

We keep saying "measure how wrong the network is." This is the job of the loss function. Choosing the right one depends on whether you are predicting numbers or categories.

5.1 Loss vs Cost — a small but important difference

The two words are often mixed up, so let us be precise. **Loss** is the error for a **single** data point. **Cost** is the **average loss over all** data points. During forward propagation we get a loss for one example; the cost tells us how the whole dataset is doing.

Lo

$$\text{MSE} = \frac{1}{n}\sum_{i=1}^{n}(y_i - \hat{y}_i)^2$$

Left: loss for one point. Right: cost = average loss over n points.

Loss functions split into two groups depending on the task: **Regression** (predicting a number, like price) and **Classification** (predicting a category, like Pass/Fail).

5.2 Regression losses

1) Mean Squared Error (MSE)

$$\text{MSE} = \frac{1}{n}\sum_{i=1}^{n}(y_i - \hat{y}_i)^2$$

MSE squares the error, so big mistakes are punished heavily. Because it is a smooth quadratic curve (a bowl shape), it is easy to do gradient descent on.

✓ Advantages

- Differentiable everywhere (smooth)
- Has one clear global minimum
- Converges faster

✗ Disadvantages

- Not robust to outliers — squaring makes a single bad point dominate the error

2) Mean Absolute Error (MAE)

$$\text{MAE} = \frac{1}{n}\sum_{i=1}^{n}|y_i - \hat{y}_i|$$

MAE takes the absolute value instead of squaring, so a single outlier does not blow up the error.

✓ Advantages

- Robust to outliers

✗ Disadvantages

- Convergence usually takes more time (needs sub-gradients at the sharp point)

3) Huber Loss — the best of both

$$L_\delta = \frac{1}{2}(y-\hat{y})^2 \text{ if } |y-\hat{y}| \le \delta\ ; \quad \delta|y-\hat{y}| - \frac{1}{2}\delta^2 \text{ else}$$

Huber loss is clever: for **small** errors it behaves like **MSE** (smooth), and for **large** errors it behaves like **MAE** (robust to outliers). The switch-over point is a **hyperparameter** δ **(delta)**. So it gives us MSE-style smoothness where errors are small, and MAE-style robustness where errors are large.

4) Root Mean Squared Error (RMSE)

$$\text{RMSE} = \sqrt{\frac{1}{N}\sum_{i=1}^{N}(y_i-\hat{y}_i)^2}$$

RMSE is simply the square root of MSE. Taking the root brings the error back to the **same unit** as the original target, which makes it easier to interpret in real-world terms.

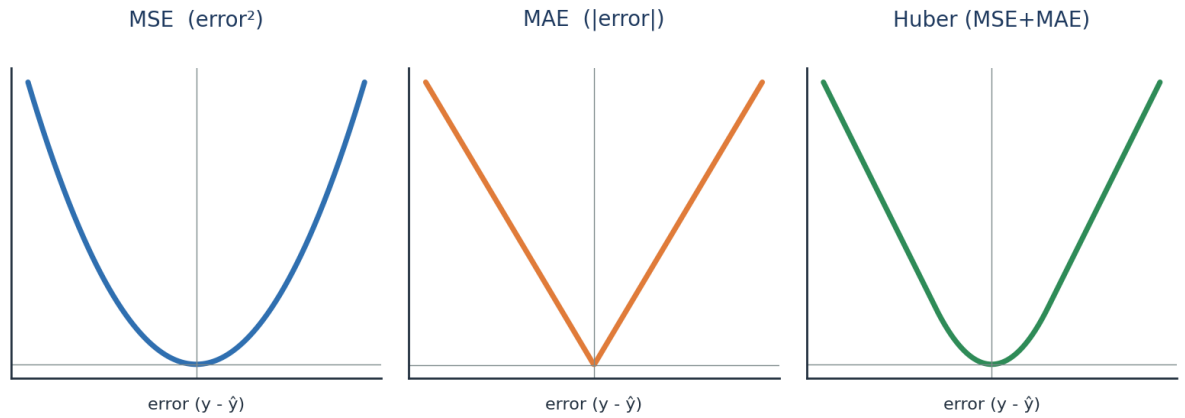


Figure 5.1 — Shapes of the three main regression losses. Notice how Huber blends the MSE bowl with the MAE V.

5.3 Classification losses — Cross-Entropy

For classification we do not use squared error; we use **Cross-Entropy**, which measures how far the predicted probability is from the true label. It comes in three flavours depending on the number of classes:

Loss	Use it when...	Label format
Binary Cross-Entropy	There are exactly 2 classes (yes/no)	0 or 1
Categorical Cross-Entropy	There are many classes	One-hot (e.g. [0,1,0])
Sparse Categorical Cross-Entropy	Many classes, but labels are integers	Integer (e.g. 2)

Binary Cross-Entropy — the classic two-class loss:

$$\text{Loss} = -\,y\log(\hat{y}) - (1-y)\log(1-\hat{y})$$

Reading it simply: if the true label **y = 1**, the loss is –log(■); if **y = 0**, the loss is –log(1–■). Either way, the closer the prediction is to the truth, the smaller the loss.

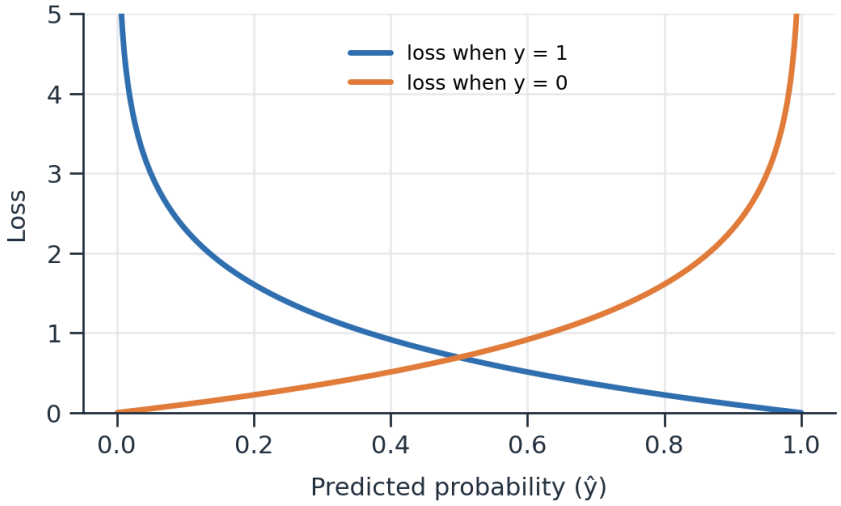


Figure 5.2 — Cross-entropy grows sharply when the prediction is confidently wrong.

Categorical Cross-Entropy — the multi-class version (used with softmax):

$$L(x_i, y_i) = -\sum_{j=1}^{C} y_{ij} \ln(\hat{y}_{ij})$$

Bridge to Part 6

We can now build a network, make a prediction, and measure the error with the right loss. The final question is the most practical one: **how exactly do we update the weights** to bring that loss down efficiently? That is the job of **optimizers**.

PART 6

Optimizers — How the Network Actually Learns

An optimizer is the exact recipe for updating weights so the loss goes down. Each optimizer here fixes a weakness of the one before it — so read them as an evolution.

6.1 The six optimizers, and how they build on each other

The notes list six optimizers. Rather than memorising them separately, notice the storyline: each new optimizer solves a specific problem left by the previous one.

#	Optimizer	The problem it solves
1	Gradient Descent	The basic downhill rule (but very resource-heavy)
2	Stochastic GD (SGD)	Fixes the resource problem (but adds noise)
3	Mini-Batch SGD	Balances speed and noise
4	SGD with Momentum	Smooths out the noise for faster convergence
5	Adagrad / RMSProp	Adapts the learning rate automatically
6	Adam	Combines Momentum + adaptive learning rate

6.2 Gradient Descent (the starting point)

First, a vocabulary reminder. One **epoch** = one full pass through the entire dataset. Plain **Gradient Descent** looks at **all** the data points, computes the average gradient, and then updates the weights **once** per epoch using the update rule from Part 3.

$$W_{new} = W_{old} - \eta \frac{\partial L}{\partial W_{old}}$$

✓ Advantages

- Convergence is stable and reliable

✗ Disadvantages

- Extremely resource-intensive — needs huge RAM/GPU to hold all data at once (imagine 1 million points)

6.3 Stochastic Gradient Descent (SGD)

SGD fixes the memory problem by going to the opposite extreme: update the weights after **every single data point**. With 1,000 points, that is 1,000 updates per epoch. This solves the resource issue, but because each step is based on just one point, the path to the bottom is **noisy** and zig-zags a lot.

✓ Advantages

- Solves the resource / memory problem

✗ Disadvantages

- Introduces a lot of noise
- Higher time complexity
- Convergence can take longer

6.4 Mini-Batch SGD (the practical middle ground)

Why choose between “all data” and “one point” when we can do something in between? **Mini-Batch SGD** updates the weights after every small **batch** (for example, 1,000 points at a time). This is the version used in practice — it gives good speed, sensible memory use, and much less noise than pure SGD.

✓ Advantages	✗ Disadvantages
• Convergence speed increases • Much less noise than SGD • Efficient use of RAM/memory	• Some noise still exists

6.5 SGD with Momentum (smoothing the path)

Mini-batch still wobbles. **Momentum** smooths the journey by remembering the direction of previous steps — like a ball rolling downhill that keeps its momentum instead of reacting to every little bump. It does this using an **Exponential Weighted Average** of past gradients (the same smoothing idea used in time-series methods like ARIMA):

$$V_t = \beta V_{t-1} + (1-\beta)\, a_t$$

Here β **(beta)**, typically around 0.95, decides how much of the past to remember. A higher β means smoother movement. The result is **less noise and faster convergence**.

✓ Advantages	✗ Disadvantages
• Reduces the noise • Quicker convergence	• Introduces an extra hyperparameter (β) to tune

6.6 Adagrad and RMSProp (adaptive learning rates)

So far the learning rate η was **fixed**. But intuitively we want **big** steps early on and **small**, careful steps as we approach the bottom. **Adagrad** (Adaptive Gradient Descent) makes the learning rate **dynamic**: it shrinks the learning rate over time based on how much a weight has already been updated.

$$W_t = W_{t-1} - \frac{\eta}{\sqrt{\alpha_t + \varepsilon}} \cdot \frac{\partial L}{\partial W_{t-1}}$$

The term α_t simply adds up the squared gradients so far, and ε is a tiny number that prevents division by zero:

$$\alpha_t = \sum_{i=1}^{t} \left(\frac{\partial L}{\partial W_t}\right)^2$$

✓ Advantages	✗ Disadvantages
• Learning rate adapts automatically — no manual tuning of η over time	• The adjusted learning rate can shrink so much it becomes almost 0, and learning stops too early

RMSProp patches exactly this weakness by using a moving average of squared gradients (instead of summing them forever), so the learning rate does not collapse to zero.

6.7 Adam — the modern default

Finally, **Adam** combines the two best ideas we just met: the **momentum** smoothing from Section 6.5 and the **adaptive learning rate** from Adagrad/RMSProp. Because it inherits the strengths of both, Adam converges quickly and reliably, which is why it is the **default optimizer** for most deep learning projects today.

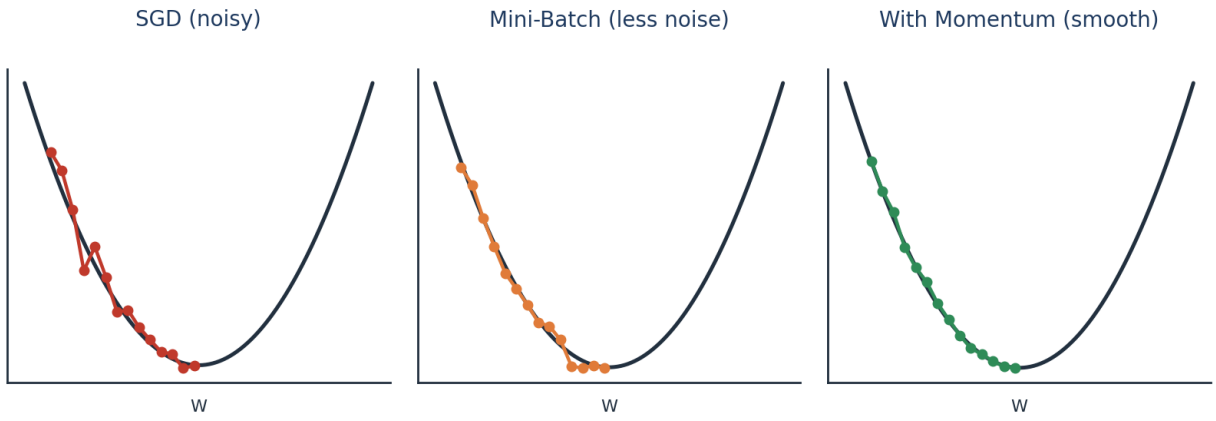


Figure 6.1 — From noisy SGD to smooth momentum: each optimizer takes a cleaner path to the minimum.

The Whole Picture, Connected

You have now travelled the entire journey. Let us tie every part back into the single story we started with.

1 · Foundations

Deep learning is a brain-inspired part of ML that thrives on big data and fast GPUs. Different data needs different networks — ANN for tables, CNN for images, RNN for sequences.

2 · The Perceptron

The smallest unit is a neuron: it takes a weighted sum of inputs, adds a bias, and applies an activation. Alone it can only draw a straight line, so we need many of them.

3 · Multi-Layer Networks

Stacked neurons make an ANN. It guesses via forward propagation, and learns via backpropagation + gradient descent, using the chain rule to share the error among all weights.

4 · Activation Functions

These add the non-linear ‘bend’. Sigmoid/Tanh cause vanishing gradients, so ReLU and its variants power hidden layers, while Softmax handles multi-class outputs.

5 · Loss Functions

They score the mistake: MSE/MAE/Huber/RMSE for regression, and Cross-Entropy for classification. This score is exactly what learning tries to minimise.

6 · Optimizers

They perform the actual weight updates. From Gradient Descent to SGD, Mini-Batch, Momentum, Adagrad/RMSProp and finally Adam — each one fixes the flaw before it.

The one-paragraph summary

A neural network **guesses** (forward propagation with activation functions), **checks its mistake** (loss/cost function), and **improves** (backpropagation with the chain rule, driven by an optimizer that follows the gradient downhill). Repeat this over many epochs and the network slowly reaches the global minima — the point where it makes the fewest mistakes. That, in essence, is how deep learning learns.

— End of Notes —